Práctica 5 de Lógica Computacional

Evaluación perezosa y acertijos

Negación por fallo, suspensión y búsqueda de soluciones

Erik Figueiral Alonso

26 de octubre de 2026

Índice

1 Negación por fallo y evaluación perezosa

2 Acertijos: describir el problema

Material de trabajo: Perezosa y Acertijos.

fail/0: fallar y volver atrás

```
?- fail.
false.
```

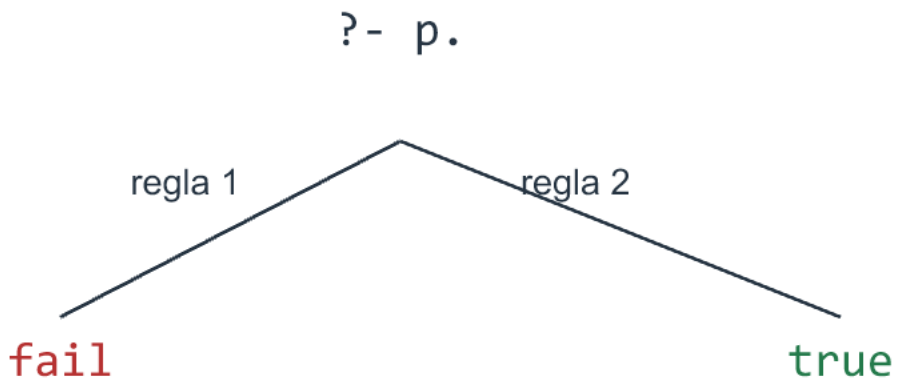
fail siempre falla. Prolog retrocede al último punto con alternativas pendientes e intenta otra opción.

Predicado	Efecto
fail	Hace fallar la rama actual.
!	Descarta alternativas dentro de su alcance.

fail no poda alternativas por sí solo.

Un fallo permite probar otra alternativa

```
p :- fail.
p.
```




```
?- p.
true.
```

Falla la primera cláusula.
La segunda todavía puede responder.

Cómo se implementa la negación por fallo

```
not(P) :- call(P), !, fail.
not(_).
```

Al intentar P	Qué hace not(P)
P tiene algún éxito	Ejecuta ! y fail: la negación falla.
P falla tras agotar su búsqueda	Llega a not(_): la negación tiene éxito.

Esquema didáctico. En SWI-Prolog se utiliza también \+ P.

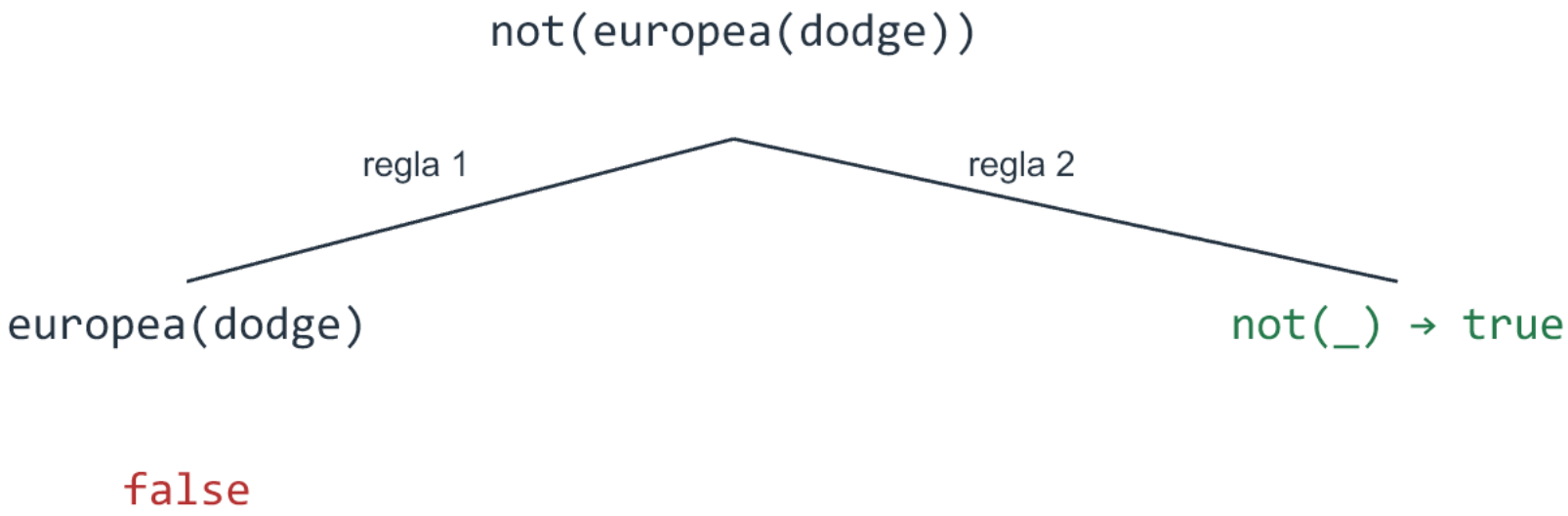
Referencia: fichero Perezosa. call(P) ejecuta el objetivo P.

Si P tiene éxito: corte y fallo



El corte descarta otras pruebas de P y la segunda regla de not.
Después, fail hace fracasar esta llamada a not.

Si P falla: la negación tiene éxito



No se llega al corte: la primera regla falla antes.
Prolog prueba la segunda regla y responde true.

No poder demostrar P no equivale siempre a demostrar su falsedad.

Negar exige saber qué estamos preguntando

Consulta	Lectura
not(europea(dodge))	No se puede demostrar que dodge sea europea.
not(europea(Marca))	No existe una Marca demostrable como europea.

Con Marca libre, Prolog busca cualquier marca europea.
Si encuentra opel, la negación falla por completo.
No enumera las marcas que no son europeas.

La interpretación depende de la información y del modo de consulta.

Coches: ¿qué responde este programa?

```
coche(dodge,caliber).
coche(opel,corsa).
coche(toyota,prius).
europea(opel).
```

```
comprar(Marca,Modelo) :-
    not(europea(Marca)), coche(Marca,Modelo).
```

```
?- comprar(X,Y).
```

Antes de ejecutarlo, decide cuándo recibe un valor Marca.

Por qué comprar(X,Y) devuelve false



coche(X,Y) ni siquiera llega a ejecutarse.

```
?- comprar(X,Y).  % false.
```

Primera solución: generar antes de comprobar

```
comprar(Marca,Modelo) :-
    coche(Marca,Modelo), not(europea(Marca)).
```

coche/2 propone	not(europea(Marca))
dodge, caliber	Tiene éxito. Se conserva.
opel, corsa	Falla. Se busca otro coche.
toyota, prius	Tiene éxito. Se conserva.

```
X = dodge, Y = caliber ;
X = toyota, Y = prius ; false.
```

Esperar al dato que falta

En una ventanilla te falta un documento. Tu trámite queda pendiente mientras se atienden otros. Cuando aportas el documento, se retoma el trámite que estaba esperando.

En Prolog	Qué sucede
Falta un valor	Se suspende un objetivo.
Se ejecutan otros objetivos	Pueden proporcionar ese valor.
Se cumple la condición	Se reactiva el objetivo pendiente.

No se espera un número fijo de pasos. Se espera una condición.

when/2: condición de activación y objetivo

```
:- use_module(library(when)).

when(nonvar(Marca), not(europea(Marca)))
```

Primer argumento: condición de activación admitida por when/2.
Segundo argumento: objetivo que se ejecutará al cumplirse.

Estado de Marca	Ejecución
Variable libre	Suspende la negación y permite continuar.
Un átomo, como dodge	Ejecuta la negación inmediatamente.

SWI-Prolog: swi-prolog.org/pldoc/man?predicate=when/2

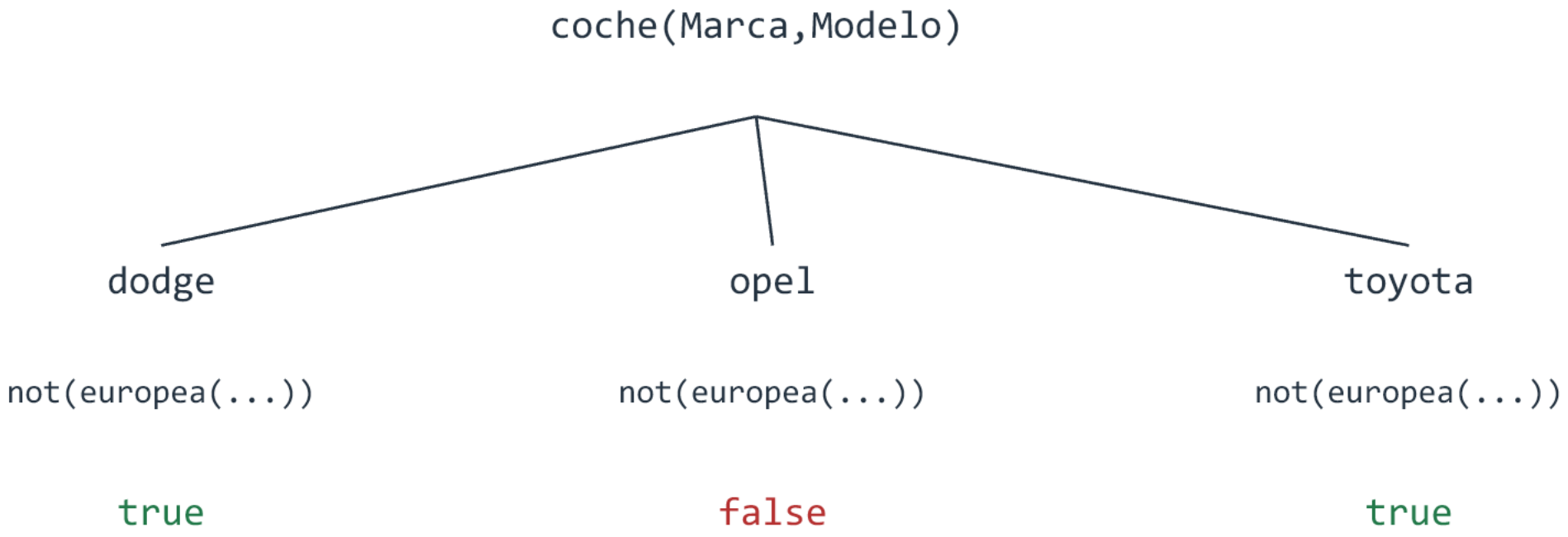
Comprar con when/2

```
puedo_comprar(Marca,Modelo) :-
    when(nonvar(Marca), not(europea(Marca))),
    coche(Marca,Modelo).
```

Paso	Situación
1. when/2	Marca libre: deja pendiente la negación.
2. coche/2	Propone dodge y caliber.
3. Se activa el objetivo	not(europea(dodge)) tiene éxito.

```
?- puedo_comprar(X,Y).
X = dodge, Y = caliber ;
X = toyota, Y = prius ; false.
```

La suspensión se reactiva en cada alternativa



El fallo de opel permite volver a coche/2 y probar toyota.

freeze/2: esperar a que una variable deje de serlo

```
puedo_comprar(Marca,Modelo) :-
    freeze(Marca, not(europea(Marca))),
    coche(Marca,Modelo).
```

Para este caso	Mismo criterio de activación
freeze(Marca, Objetivo)	Espera a que Marca no sea variable.
when(nonvar(Marca), Objetivo)	Espera a que nonvar(Marca) sea cierto.

Si Marca ya tiene un valor, el objetivo se ejecuta en ese momento.

SWI-Prolog: swi-prolog.org/pldoc/man?predicate=freeze/2

Suspender no equivale a resolver

```
?- freeze(X, X = permitido).
```

Si ningún objetivo proporciona un valor a X, la condición queda pendiente. La consola puede mostrar un objetivo residual.

```
?- freeze(X, X = permitido), X = permitido.
X = permitido.

?- freeze(X, X = permitido), X = prohibido.
false.
```

nonvar/1 no significa «sin ninguna variable»

Término	nonvar/1	ground/1
Marca libre	false	false
dodge	true	true
marca(X)	true	false

Para nuestras marcas, que son átomos, nonvar(Marca) basta.
Si el objetivo necesita un término totalmente instanciado,
when(ground(Termino), Objetivo) expresa esa condición.

SWI-Prolog: condiciones de when/2.

Ejercicio: soltero/1

```
casado(juan).
varon(juan).
varon(pepe).

soltero(P) :- not(casado(P)), varon(P).
```

```
?- soltero(X).
```

Explica el resultado y escribe tres versiones:
cambiando el orden, con when/2 y con freeze/2.

Modelo simplificado del fichero: se usa solo esta base de hechos.

Solución: tres formas de consultar soltero/1

```
soltero_orden(P) :-
    varon(P), not(casado(P)).

soltero_when(P) :-
    when(nonvar(P), not(casado(P))), varon(P).

soltero_freeze(P) :-
    freeze(P, not(casado(P))), varon(P).
```

La versión original devuelve false: casado(X) encuentra juan.
Las tres versiones corregidas obtienen X = pepe.

Otro problema: recursividad por la izquierda

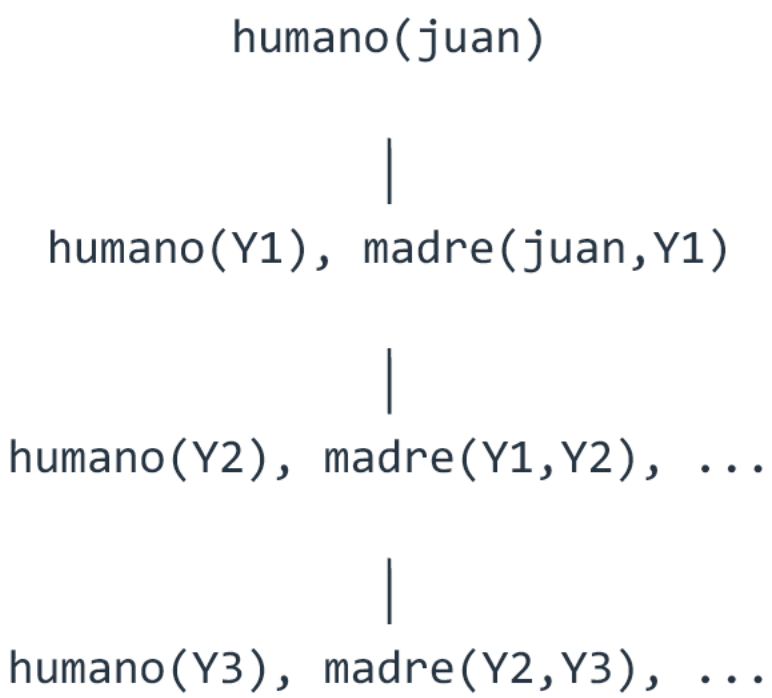
```
humano(X) :- humano(Y), madre(X,Y).
humano(elena).
madre(juan,elena).
```

Aquí madre(Hijo, Madre) relaciona al hijo con su madre.
La intención: alguien es humano si su madre es humana.

```
?- humano(juan).
```

Con este orden, la búsqueda no llega a responder.

El árbol crece sin consultar madre/2



Siempre se elige primero otra llamada recursiva. No avanza la búsqueda.

Rama izquierda del árbol SLD. Las alternativas del axioma quedan pendientes.

Cambiar el orden de los objetivos

```
humano(X) :- madre(X,Y), humano(Y).
humano(elena).
madre(juan,elena).
```

Consulta: humano(juan)	Resultado intermedio
madre(juan,Y)	Y = elena
humano(elena)	El axioma permite responder true.

Ahora la llamada recursiva recibe una persona concreta.

Poner el axioma primero no elimina todo bucle

```
humano(elena).
humano(X) :- humano(Y), madre(X,Y).
```

Consulta	Comportamiento con estos hechos
humano(juan)	Encuentra una primera respuesta true.
humano(X), pidiendo todas	Obtiene elena y juan; después sigue en un bucle.

Mejora el acceso a algunas respuestas, pero mantiene la recursividad izquierda.

Retrasar la llamada recursiva

```
es_humano(X) :-
    freeze(Y, es_humano(Y)), madre(X,Y).
es_humano(elena).
```

Al preguntar es_humano(juan)	Qué sucede
Y es variable	es_humano(Y) queda pendiente.
madre(juan,Y)	Instancia Y = elena y reactiva la llamada.
es_humano(elena)	La regla no encuentra madre; el axioma tiene éxito.

Fichero Perezosa. También: when(nonvar(Y), es_humano(Y)).

Volver con información nueva

Un guardia te envía a otro y ese te devuelve al primero.
Si vuelves sin información nueva, repites la misma pregunta.

Con suspensión, se deja pendiente humano(Y) mientras falta Y.
madre(X,Y) aporta el nombre. Se vuelve a preguntar por esa persona.

```
Y libre  →  llamada pendiente
Y = elena  →  es_humano(elena)
```

freeze/2 no resuelve cualquier ciclo: aquí ayuda a cambiar el orden efectivo.

Acertijos: describir relaciones y restricciones

Elegimos cómo representar el escenario. Traducimos las pistas a relaciones y condiciones que una solución debe cumplir.

El intérprete busca asignaciones y retrocede cuando una pista no se cumple. El orden de los objetivos sigue influyendo en la búsqueda.

Hoy desarrollamos El envenenador. Después: Ocupaciones y Las cinco casas.

El envenenador: seis frascos, siete pistas

El Dr. Bacterio tiene seis frascos: rojo, naranja, amarillo, verde, azul y violeta. Algunos contienen veneno.

En cada par: uno tóxico y el otro no	Pista
violeta y azul	1
rojo y amarillo	2
azul y naranja	3

¿Qué frascos contienen veneno?

Enunciado original: Acertijos / El envenenador.

El envenenador: las pistas restantes

En cada par: al menos uno no es tóxico	Pista
violeta y amarillo	4
rojo y naranja	5
verde y azul	6

7. El frasco rojo no es venenoso.

«Al menos uno» admite que los dos sean no tóxicos.

Ejercicio: representar las siete pistas

```
uno_toxico_otro_no(Color1,Color2)
uno_no_toxico(Color1,Color2)
no_toxico(Color)
```

Escribe los hechos que traducen las pistas.
Después piensa qué puedes deducir al conocer
que uno de los frascos de un par es tóxico o no tóxico.

No confundas «uno de cada tipo» con «al menos uno no tóxico».

Solución: los hechos del envenenador

```
uno_toxico_otro_no(violeta,azul).
uno_toxico_otro_no(rojo,amarillo).
uno_toxico_otro_no(azul,naranja).

uno_no_toxico(violeta,amarillo).
uno_no_toxico(rojo,naranja).
uno_no_toxico(verde,azul).

no_toxico(rojo).
```

Código de referencia: Acertijos / El envenenador.

Las reglas del código de referencia

```
no_toxico(X) :-
    (uno_no_toxico(X,Y); uno_no_toxico(Y,X)),
    toxico(Y).

toxico(X) :-
    (uno_toxico_otro_no(X,Y);
     uno_toxico_otro_no(Y,X)), no_toxico(Y).
```

Se contemplan ambos órdenes del par. El punto y coma expresa «o».
Cada consulta intenta demostrar el estado de un frasco.

no_toxico/1 es una relación explícita. No está definido como not(toxico(X)).

findall/3: reunir las respuestas de una consulta

```
findall(Plantilla, Objetivo, Lista)
```

Busca las soluciones de Objetivo por retroceso. Por cada éxito,
recoge una copia de Plantilla en Lista, conservando posibles repetidos.

```
?- findall(Marca,coche(Marca,_),L).
L = [dodge,opel,toyota].

?- findall(X,toxico(X),L).
L = [azul,amarillo].
```

SWI-Prolog: swi-prolog.org/pldoc/man?predicate=findall/3. Sin soluciones: L = [].

Una alternativa: buscar soluciones completas

findall(X,toxico(X),L) reúne demostraciones individuales.
Por sí mismo no exige que L sea una asignación conjunta compatible.

```
[Rojo,Naranja,Amarillo,Verde,Azul,Violeta]
```

Usaremos 0 para no tóxico y 1 para tóxico.
Una respuesta asignará un estado a los seis frascos y cumplirá las siete pistas simultáneamente.

Seis estados binarios: 64 asignaciones posibles antes de aplicar las pistas.

Ejercicio: comprobar una asignación completa

```
resolver_estados([R,N,A,V,Az,Vi])
```

Genera un 0 o un 1 para cada frasco y aplica las siete pistas.
Cada respuesta debe representar todos los frascos a la vez.

Tipo de pista	Condición sobre dos estados X e Y
Uno tóxico y otro no	X =\= Y
Al menos uno no tóxico	X + Y =< 1

Consulta todas las respuestas para saber si existe otra combinación.

Solución: generar y comprobar las siete pistas

```
estado(0).
estado(1).

resolver_estados([R,N,A,V,Az,Vi]) :-
    estado(R), estado(N), estado(A),
    estado(V), estado(Az), estado(Vi),
    Vi =\= Az, R =\= A, Az =\= N,
    Vi + A =< 1, R + N =< 1, V + Az =< 1,
    R = 0.
```

Primero se generan valores. Las comparaciones aritméticas reciben números.

Cada respuesta es una combinación compatible

```
?- findall(E,resolver_estados(E),Soluciones).
Soluciones = [[0,0,1,0,1,0]].
```

Frasco	Estado
rojo / naranja / verde / violeta	0: no tóxicos
amarillo / azul	1: tóxicos

Hay una única asignación completa que satisface las siete pistas.
Si hubiera varias, findall/3 devolvería una lista por cada asignación.

Después: dos acertijos disponibles en Moovi

1 Ocupaciones

2 Las cinco casas

Trabajad primero Ocupaciones y después Las cinco casas.
Los enunciados y el código están en Moovi.